

10th International Conference of Information and Communication Technology (ICICT-2020)

ProbWalk: A random walk approach in weighted graph embedding

Xiaohua Wu, Hong Pang, Youping Fan, Yang Linghu, and Yu Luo*

School of Information and Software Engineering, University of Electronic Science and Technology of China, Chengdu, 610054, China

Abstract

There are many weighted graphs in the real-world networks, such as social networks, communication networks, citation networks, etc. Along with successful application of deep learning in graph embedding, we study how to embed weight graph, because weights on the edges also play an important role in the graph. We propose a novel algorithm called ProbWalk, which take advantage of edge weights and convert the weights into transition probabilities. Our proposed method specifies the strategy of sampling the surrounding vertices by weights and generate the random walk for graph embedding according to transition probability. We evaluate our methods on tasks including multi-label classification and link prediction. Experimental results show that our method performs better than competed method on several weighted graph datasets.

© 2021 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the 10th International Conference of Information and Communication Technology.

Keywords: graph embedding, machine learning, network representation learning, weighted graph embedding;

1. Introduction

Weighted graph has been widely extracted from ubiquitous networks, and is not only important in mathematics but also significant in our daily life. For example, communication networks can weigh the frequency of communication on edges^[1], and the weight may be a measure of the cooperation times between two authors in citation networks^[2]. Weighted graph embedding aims to obtain the low-dimensional vector of each vertex in the weighted graph. And it is useful to conduct many machine learning applications at graph mining and network analysis tasks, such as node classification^[3], link prediction^[4] and network visualization^[5].

Most existing methods of graph representation learning inspired by the emergence of representation learning^[6]. Specifically, DeepWalk^[7] generates truncated random walks as sequences from unweighted graph by uniform

* Corresponding E-mail address: luoyu@uestc.edu.cn

sampling method. Therefore, it only concerns that whether there is a connection between two nodes. Moreover, node2vec^[8] introduces two parameters to control the graph structure with sampling, which import external parameters to search the structure of graph with Breadth-first sampling (BFS) and Depth-first sampling (DFS) rather than the original weights in the graph. Although, these methods are performed well on general graph or network. But it are not satisfied for learning the representation of weighted graph. Because most existing graph embedding approaches mainly focus on how to embed a binary-connected graph or network into a low-dimensional space. It means that they ignore the weight on edge in graph representation learning. In LINE^[9], it mentioned that their proposed model is able to apply to weighted graph. It directly takes the first order and second-order proximities of vertices to preserve the structure of networks. However, it does not explicitly leverage the weights on graph when learn the representation of graph. In detail, the weight of edge will be multiplied by the gradient which cause a high variance problem and is hard to find a good learning rate. Consequently, it is a challenging problem that how to leverage the weight of edge in graph embedding.

To address the problem, we focus on the weighted graph representation in this paper, which aim to embed weighted graph into low-dimensional dense vector space avoiding the weakness of sparsity. Based on the assumption that the weight on edge can be converted as transition probability, we propose a new weighted graph embedding model, which is coined as ProbWalk. Intuitively, our approach preserves the properties of vertices according to their surrounding vertices and the weights between them. The model generates a sequence of vertices based on the weight distribution of one vertex and its surrounding vertices. We assume that vertices share the similar surrounding vertices will also be similar in vector space. Therefore, we utilize the weights on graph to construct a probability distribution among surrounding vertices. Next, we extend the model from natural language processing to learn representation of each vertex and its surrounding vertices in the graph. Meanwhile, we adopt a new objective function using stochastic gradient descent for optimization to train our model.

There is an illustrative example of weighted graph embedding showed in Figure 1. Our method can learn the weighted graph structure into low-dimensional representation. In Figure 1 (a) it is a transportation network of Chinese mainland cities which data come from Baidu Migration Project. The vertices in the graph represent cities. And the difference in edge thickness represents the difference in weight values. Through weighted graph embedding, our proposed method learns the structure of a transportation network and weights on edges. Figure 1 (b) shows the position of vertices in a two-dimensional space with the PCA dimension reduction. It means that the cities in close geographical location and frequent population transportation are also close in the representation space.

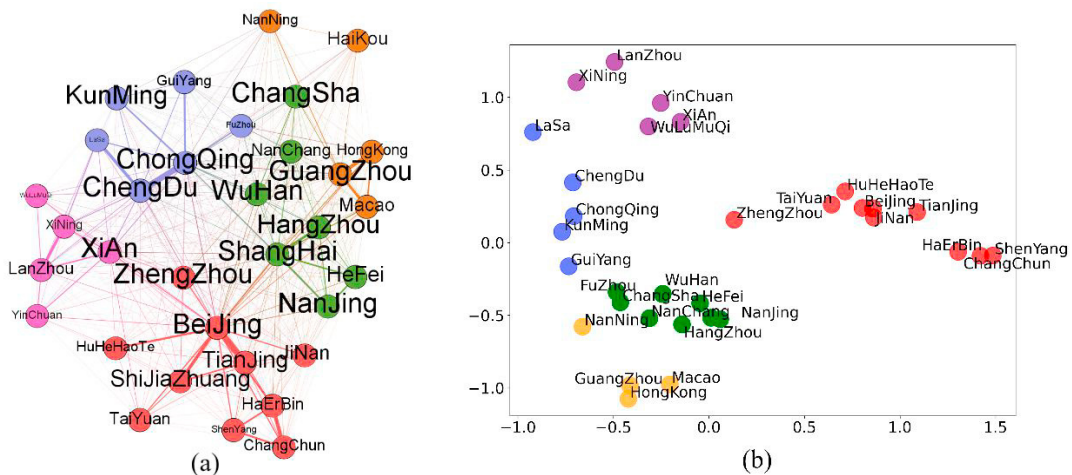


Fig. 1. (a) The transportation network of China mainland cities; (b) The representations of vertices in 2-dimensional space with PCA dimension reduction.

The rest of the paper is organized as follows. In Section 2, we introduce the problem definition about weighted graph embedding. We present ProbWalk model in details in Section 3. In Section 4, we conduct multi-label

classification and link prediction experiments. Afterwards, we explain the experimental results. Finally, we close with our conclusions in Section 5.

2. Problem Definition

We formally define the problem of weighted graph embedding. Given a weight graph $G = (V, E, W)$, where V is the set of vertices, E represents the edges between vertices and W represents the weighted value on the edges. To learn a mapping function $f_G: V \rightarrow \mathcal{R}^d$, which $d \ll |V|$. Each vertex is embedded into a space $\mathcal{R}^d$. The representation of vertex in this space is also called learned feature. The vertex feature which learned by graph embedding methods have many advantages. Firstly, the dimension of feature is low and could be controlled. Secondly, the value of feature is continuous and a continuous representation could benefit downstream machine learning applications.

3. Our Model

In this section we propose a novel method named ProbWalk that leverages the weights on edges in the graph by computing a walk probability from surrounding vertices' weights distribution. And we design a new objective function for our model to learn the embedding of each vertex. This model generates a sequence of walks by the transition probability based on the weights in the graph. We calculate the transition probability based on the original weights on edges in the graph or network. Obviously, it is not uniform distribution because the weights on the network are different in many situations. Therefore, the transition probability distribution depends on the weights on the edges between a vertex and its neighborhoods. Hence, we propose the ProbWalk method to learn vertex representation in weighted graph embedding. Formally, consider a probability walk started at vertex u as R_u . Hence, we can generate a sequence of vertices $R_u = \{u_1, u_2, \dots, u_l\}$, which l is the length of this walk. According to the weight $w_{u,v}$ on the edge between two vertices u and v , we can calculate the walk probability from vertex u to one of its neighbors. It is defined as $P(u, v) = \frac{w_{u,v}}{\sum_{v \in N_u} w_{u,v}}$ where v is one of the neighbors of vertex u .

To extend skip-gram model in weighted graph embedding, we process vertices in the graph as words in natural language processing. Therefore, we assume that a vertex has close distance with the other connected vertex in latent vector space. Accordingly, we can define the joint probability between the start vertex v and its connected surrounding vertices $C = \{v_i | i = 1, \dots, k\}$ in Equation (1).

$$P(v_1, \dots, v_k | v) = \prod_{v_c \in C} \frac{e^{u_i \cdot u}}{\sum_{i=1}^n e^{u_i \cdot u}} \quad (1)$$

In Equation (1) n is the number of all vertices and u is the latent representation of v . To maximize the conditional probability, we define an objective function or loss function as $E = - \sum_{c=1}^{|C|} \sigma(u_i \cdot u) + C \cdot \log \left[\sum_{i=1}^n e^{u_i \cdot u} \right]$ where $\sigma(x) = 1/[1 + \exp(-x)]$ is the sigmoid function. Therefore, we need to minimize the objective to learn the representation u . According to the formation of objective function, it requires to calculate the summation of all vertices' dot production with vertex v . Therefore, we use negative sampling as an effective method to compute the loss function. Therefore, we approximate the objective as Equation (2).

$$E = - \sum_{C=1}^{|C|} \sigma(u_i \cdot u) - \sum_{i=k}^k E_{v_k \in P_n(v)} \log [e^{u_k \cdot u}]. \quad (2)$$

$P_n(v)$ denote the set of negative vertices which is not connected to v in Equation (2). We set the value of k according to the size of dataset follows the conclusion at word2vec^[10], which is set $k = [2, 5]$ for large dataset and $k = [5, 10]$ for small dataset. We optimize the well-designed objective using stochastic gradient descent (SGD).

4. Experiments

We test our ProbWalk for weighted graph embedding on two main tasks: multi-label classification and link prediction. We use two benchmark weighted network datasets: Wiki-POS^[8] and Airport^[11]. We compare our method with DeepWalk which utilize uniform random walks to reconstruct a whole graph in both tasks. Additionally, we conduct a visualization analysis experiment on the transportation networks of Chinese mainland cities from Baidu Migration Project.

4.1. Datasets

The experiments in this paper use two datasets summarized as follows. Wiki-POS dataset is the word co-occurrence network from the Wikipedia corpus. Hence, a word represents a node in this network and the weight on the edge is the times of word cooccurrence. Statistically, this graph has 4,777 vertices, 184,812 edges and 40 different labels. Therefore, we conduct multi-label classification experiment on this dataset. In addition, Airport dataset records the number of passengers traveling between airports in US, which is used for the link prediction task. The numbers of nodes and edges about the two datasets are showed in Table 1.

Table 1. The datasets used in both experiments.

Dataset Name	Node numbers	Edge numbers
Airport	500	5960
Wiki-POS	4,777	184,812

4.2. Multi-label classification

We experiment our methods on the task of multi-label classification. In the multi-label classification, vertices might have more than one label. For brevity, we check how the top-ranking predictions match with the true labels. And we use two common measures Micro-F1 and Macro-F1 to evaluate the classification performance as previous multi-label classification work does^[12]. Specifically, we randomly split the dataset into two parts as train set and test set with different portion, which is from 10 % to 90 %. In order to avoid random, we repeat our experiment 10 times and report the average performance in terms of both measures. For all methods, we choose a one-vs-rest logistic regression implemented by LibLinear^[13] to predict the most probable labels. The parameters are set to be the same as in DeepWalk. In detail, we set the number of embedding dimensions is 128, walk length is 40, the number of walks for each vertex is 80. The window size of Skip-gram is also the same, which is set to 10.

For multi-label classification task, the result of comparison between two methods draws in Figure 2. It is evident that ProbWalk perform consistently better than DeepWalk. Because utilizing the weights on edges in walking stage make the frequency of node appearance in corpus is more reasonable. In fact, the frequency is a latent representation of the similarity of two vertices. Therefore, our method outperforms DeepWalk in the multi-label classification task. Additionally, we can discover that the ProbWalk algorithm distinguish from DeepWalk's uniform walk exploration. With the help of normalization of weights on neighbors, the ProbWalk have the ability to choose the most probable walk in a sequence of sampling. Hence, our method outperforms DeepWalk by at least 8% with respect to Micro-F1. In other words, our method achieves a gain of 16% gain over DeepWalk in Micro-F1 scores. Similarly, the performance of our method is also better than DeepWalk in Macro-F1 scores.

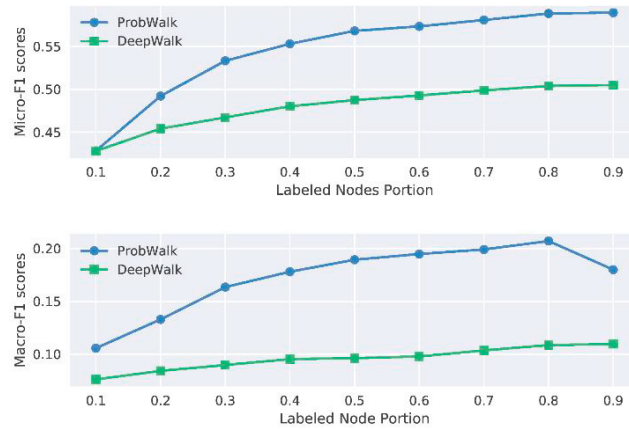


Fig. 2. Performance comparison of two methods in Wiki-POS dataset with different labeled node portions.

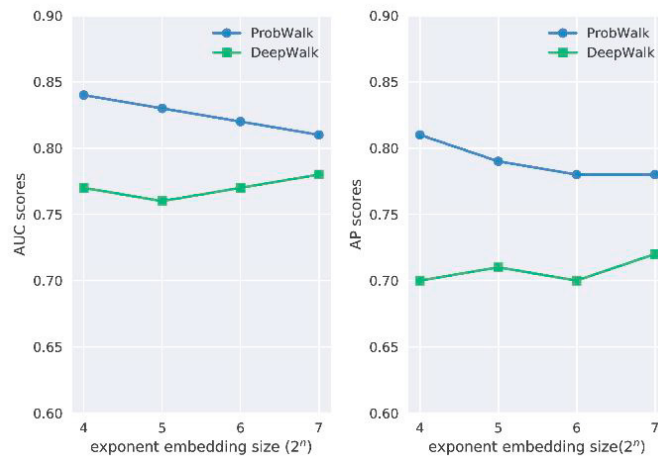


Fig. 3. The performance of our method and DeepWalk with different embedding dimension size in Congress dataset.

4.3. Link Prediction

In link prediction, we would like to predict the missing edges on networks where parts of the edges have been hidden. At the stage of training, we can obtain the representation of each vertex using the methods of graph embedding. We use the obtained representations to conduct the link prediction task and compare our method against Deepwalk. The parameters settings for both methods are totally equal, i.e. 10 random walks of length, walk 80 times starting from each node and a context size of 10 in word embedding setting. The embedding dimension of both methods are changed from 16 to 128 in this experiment. We gradually increase the size of embedding dimension from 16 to 128 and draw the result in Figure 3. We can observe that our method outperforms DeepWalk in this result with gain up to 3.7% to 13.5% respectively in both the AUC scores and AP scores. When the size of embedding dimension gradually decreases, the result scores in our methods become higher while the scores got by DeepWalk go down. It demonstrates that our method is more effective than DeepWalk because our method obtains higher scores than DeepWalk with less information.

4.4. Visualization for transportation networks

To explain the meaning of our method in visualization application, we conduct an additional experiment at transportation network and analyze the learned representation in vector space. We draw the transportation network layout and the scatter of vertices in 2-dimensional space which showed in Figure 1. This transportation network is a weighted graph and its weight stands for the degree of population transportation between two cities. In order to prove that our method utilize the information of weights in the weighted graph, we compare the distribution of vertices frequency between our method and DeepWalk. In Table 2 it directly records the statistic of two vertices sampling distributions whose corpus generated by ProbWalk and DeepWalk. Judging from the difference, it indicates that our method has taken into weights in the graph by the distribution. The means of vertices sampling times of both methods are equal, because the parameter of walk length is set to 10 and the times of sampling is set to 20 in both methods. However, the standard deviations are different. Additionally, the max count times of our method is greater than DeepWalk. It means that our method sample vertices by weights to put this information in consideration. While DeepWalk just randomly walk to capture the structure of graph which caused a uniform distribution of vertices sampling times. Therefore, it demonstrates that our method actually can leverage the weights on graph when the corpus generated by the probabilistic approach.

Table 2. The statistics of nodes sampling times distribution generated by our method and DeepWalk.

Statistics	mean	std	max	min
ProbWalk	200	83.89	470	94
DeepWalk	200	30.34	250	137

5. The Conclusions

In this paper, we deal with the problem of weighted graph embedding since the weights on edges are useful for graph representation learning. We propose a novel algorithm named ProbWalk, which is able to sample a sequence of vertices in the graph based on the weights on edges. Afterward, the output representation of weighted graph can be used in a variety of machine learning applications such as multi-label classification, link prediction, and visualization. We demonstrate the performance of our method on multi-label classification and link prediction tasks. It shows that our method provide Micro-F1 scores up to 8% higher than competing method in classification, and achieve better performance on link prediction at AUC scores. In future work, we would like to propose new method for dynamic weighted graph embedding since the weights in the graph might evolve with time.

Acknowledgement

This work was supported by the grant of Sichuan Science and Technology Program (No. 2018GZDZX0042), China.

References

1. LESKOVEC, Jure, et al. Community structure in large networks: Natural cluster sizes and the absence of large well-defined clusters. *Internet Mathematics*, 2009, 6.1: 29-123.
2. TANG, Jie, et al. Arnetminer: extraction and mining of academic social networks. In: *Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*. 2008. p. 990-998.
3. BHAGAT, Smriti; CORMODE, Graham; MUTHUKRISHNAN, S. Node classification in social networks. In: *Social network data analytics*. Springer, Boston, MA, 2011. p. 115-148.
4. LIBEN - NOWELL, David; KLEINBERG, Jon. The link - prediction problem for social networks. *Journal of the American society for information science and technology*, 2007, 58.7: 1019-1031.
5. Herman I, Melançon G, Marshall M S. Graph visualization and navigation in information visualization: A survey. *IEEE Transactions on visualization and computer graphics*, 2000, 6(1): 24-43.
6. Bengio Y, Courville A, Vincent P. Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence*, 2013, 35(8): 1798-1828.

7. Perozzi B, Al-Rfou R, Skiena S. Deepwalk: Online learning of social representations. Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining. 2014: 701-710.
8. Grover A, Leskovec J. node2vec: Scalable feature learning for networks. Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining. 2016: 855-864.
9. Tang J, Qu M, Wang M, et al. Line: Large-scale information network embedding. Proceedings of the 24th international conference on world wide web. 2015: 1067-1077.
10. Mikolov T, Sutskever I, Chen K, et al. Distributed representations of words and phrases and their compositionality. Advances in neural information processing systems. 2013: 3111-3119.
11. Colizza V, Pastor-Satorras R, Vespignani A. Reaction–diffusion processes and metapopulation models in heterogeneous networks. *Nature Physics*, 2007, 3(4): 276-282.
12. Tang L, Liu H. Relational learning via latent social dimensions. Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining. 2009: 817-826.
13. Fan R E, Chang K W, Hsieh C J, et al. LIBLINEAR: A library for large linear classification. *Journal of machine learning research*, 2008, 9(Aug): 1871-1874.